

Patrons de conception et interactions humain-machine

Alexis Teurterie

Théo Genestier

Gaël Peschet



Contents

1	Introduction au projet	2
2	Patterns de conception utilisés	3
2.1	Détails des patterns	4
2.1.1	Chain of Responsibility — <i>CasterLink</i>	4
2.1.2	Template — <i>CasterLink</i>	4
2.1.3	Prototype — <i>MFSshape</i>	4
2.1.4	Look&Feel / Factory	4
2.1.5	Stratégie — <i>Evaluation</i>	4
2.1.6	MVC	4
2.1.7	Observateur — <i>Listeners</i>	5
2.1.8	Command	5
2.1.9	State	5
3	Developpement du projet	6
4	Conclusion du projet	13

1 Introduction au projet

Afin de conclure le programme de "Patrons de conception et interactions humain-machine", nous avons eu pour projet de créer un jeu de mémorisation de formes.

Ce devoir de Contrôle Continu était à réaliser en groupe de 3 étudiants. Une partie du temps de travail des derniers TPs y était consacrée mais le travail hors de ces heures était nécessaire.

Le but de ce projet était, évidemment, de faire un jeu mais avec certaines conditions :

Il devait être facile à comprendre, facile à maintenir et à faire évoluer et robuste ainsi que axé autour des design patterns. De plus, il devait également être testé.

Nous souhaitons une réalisation intégralement MVC, avec un modèle totalement indépendant de l'interface graphique. En particulier, le jeu devra être jouable en ligne de commande (par affichage sous forme de `System.out.println(...)`).

La partie interface ne constituera donc, comme toujours en MVC, qu'une partie Vue-Contrôleur supplémentaire venant se greffer sur le modèle.

2 Patterns de conception utilisés

Ce projet met en œuvre plusieurs **design patterns** classiques, chacun servant un objectif précis en lien avec la structuration, la modularité ou l'évolutivité de l'application.

Résumé des patterns utilisés

Pattern	Composant associé	Rôle
Chain of Responsibility	CasterLink	Crée la vue associée à une forme, en passant la requête entre différents "casters".
Template	CasterLink	Encadre le processus de conversion d'un objet vers sa représentation visuelle.
Prototype	MFSShape	Permet le clonage des formes pour en créer des copies indépendantes.
Look & Feel / Factory	Boutons, labels	Permet de générer différents styles pour l'interface utilisateur via des fabriques spécialisées.
Stratégie	Evaluation	Fournit différentes manières d'évaluer les différences entre deux conteneurs, normalisées entre 0 et 100.
MVC	Vue / Modèle	Gère l'interface graphique en séparant clairement les rôles du modèle, de la vue et du contrôleur.
Observateur	Listeners	Met à jour dynamiquement les vues en fonction des changements dans les conteneurs de formes.
Command	Actions sur formes	Encapsule les actions sur les formes (modification, annulation, etc.).
State	Modes de l'application	Gère les modes de l'éditeur (Création, Mouvement, Suppression, Redimensionnement).

2.1 Détails des patterns

2.1.1 Chain of Responsibility — *CasterLink*

Ce pattern permet à plusieurs objets de traiter une requête sans que le client sache lequel la prendra en charge. Ici, les objets de type `CasterLink` sont chaînés pour convertir une forme abstraite en sa représentation graphique. Cela rend le système extensible et souple.

2.1.2 Template — *CasterLink*

Le pattern Template permet de définir le squelette d'un algorithme dans une classe abstraite, tout en laissant certaines étapes à des sous-classes. C'est exactement ce que fait `CasterLink`, en proposant une structure commune à tous les convertisseurs de formes.

2.1.3 Prototype — *MFShape*

Utilisé pour cloner des formes, ce pattern permet de dupliquer des objets complexes sans dépendre de leur classe concrète. `MFShape` implémente une méthode de clonage conforme à ce modèle.

2.1.4 Look&Feel / Factory

Pour assurer une cohérence visuelle tout en permettant la personnalisation, les boutons et labels sont créés à l'aide de fabriques. Ces dernières génèrent les éléments en fonction d'un style choisi (moderne, classique, etc.).

2.1.5 Stratégie — *Evaluation*

Différentes stratégies d'évaluation des différences entre conteneurs sont encapsulées dans des objets `Evaluation`. On peut facilement remplacer ou étendre le système avec de nouveaux critères sans modifier le code existant.

2.1.6 MVC

L'architecture suit le pattern MVC, où le **modèle** représente les formes et leurs propriétés, la **vue** affiche leur représentation, et le **contrôleur** gère les interactions utilisateur. Ce découplage améliore la maintenabilité.

2.1.7 Observateur — *Listeners*

Les composants graphiques sont informés en temps réel des modifications via des listeners. Certains sont spécifiques aux conteneurs de formes, permettant des interactions fines (comme la mise à jour des liaisons).

2.1.8 Command

Chaque action utilisateur (ajout, suppression, modification de forme) est encapsulée dans un objet Commande. Cela rend possible la gestion d'un historique pour l'annulation/refaire des actions.

2.1.9 State

L'éditeur change de comportement selon le mode actif (création, déplacement, suppression, redimensionnement), en suivant le pattern State. Chaque état implémente un comportement distinct pour gérer les événements.

3 Développement du projet

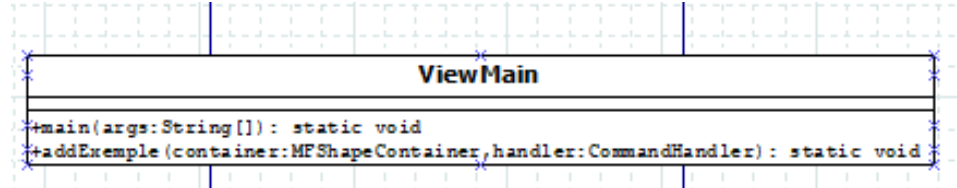
Le code du projet se divise en plusieurs dossiers :

- "main"
- "test"

Le dossier "tests" contient des tests unitaires permettant la vérification du bon fonctionnement des formes et de leur container. Le dossier "main" est le plus complet et est lui-même réduit en plusieurs sous-dossiers :

- "main"
- "model"
- "view"

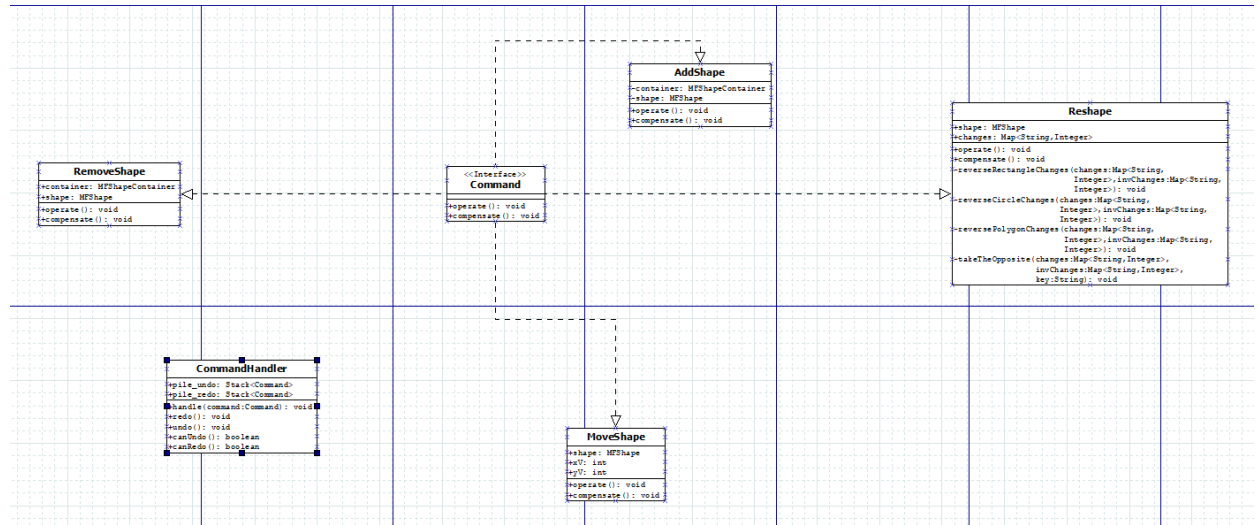
Dans le dossier "main", on retrouve un fichier pour obtenir le résultat de l'exécution de plusieurs tests ainsi qu'un fichier pour obtenir une vue avec des formes déjà placées et modifiées



Le dossier "model" comporte 5 sous-dossiers :

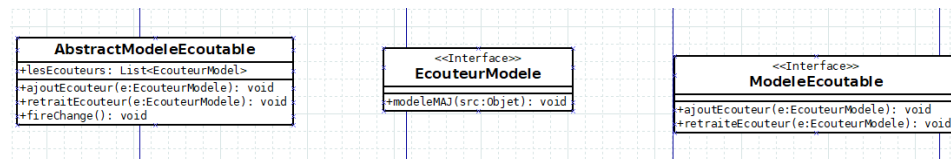
- "command"
- "observer_pattern"
- "shape"
- "strategy"
- "utils"

”command” est le dossier pour les fichiers qui correspondent aux instructions qui sont en lien aux formes : les ajouter, les bouger, les enlever, les redimensionner.



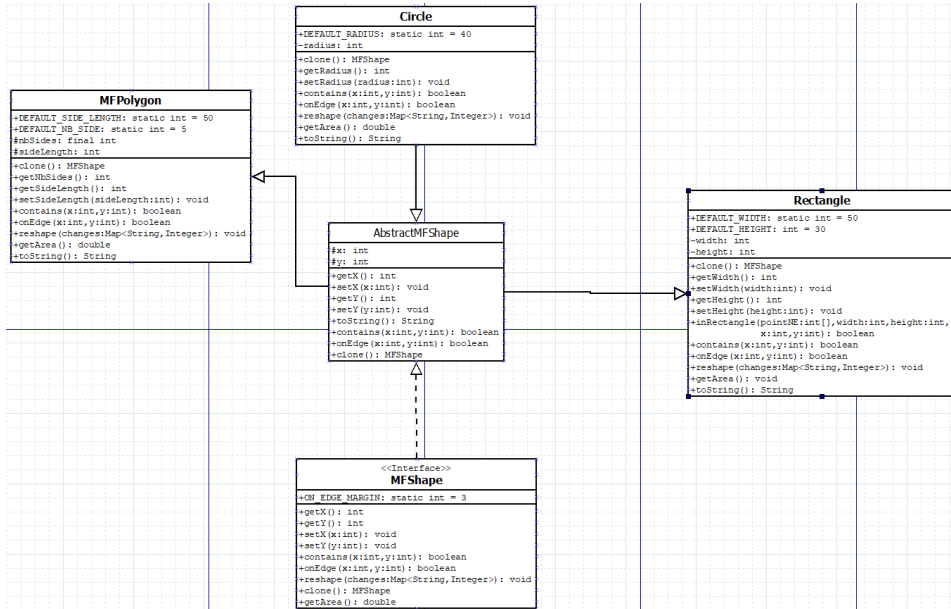
”observer_pattern” contient les fichiers pour faire fonctionner les écouteurs:

Une classe abstraite qui appelle les méthodes de ces 2 interfaces en initialisant une liste de Modele Ecoutable. Une interface pour mettre à jour le modele. Une interface pour ajouter et retirer les ”Listener”.

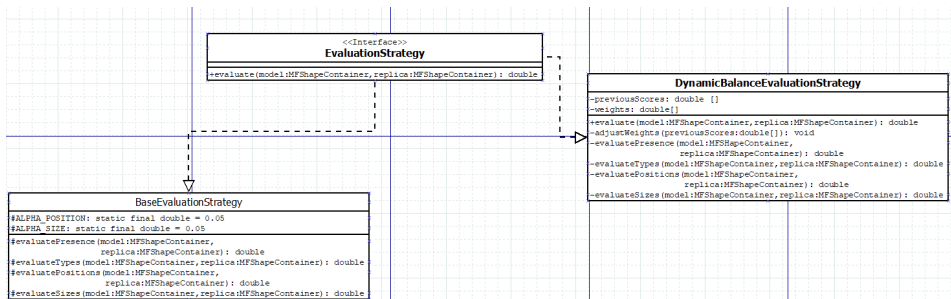


Ainsi qu’un dossier qui contient des fichiers pour faire la même chose mais au niveau des containers

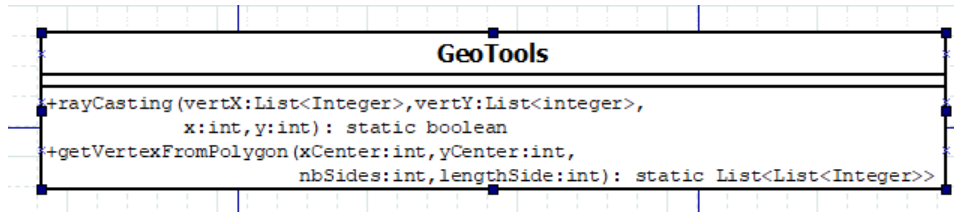
”shape” a tous les fichiers qui representent les formes (cercle, rectangle et polygones)



”strategy” contient toutes les strategies d’évaluation que nous avons développées

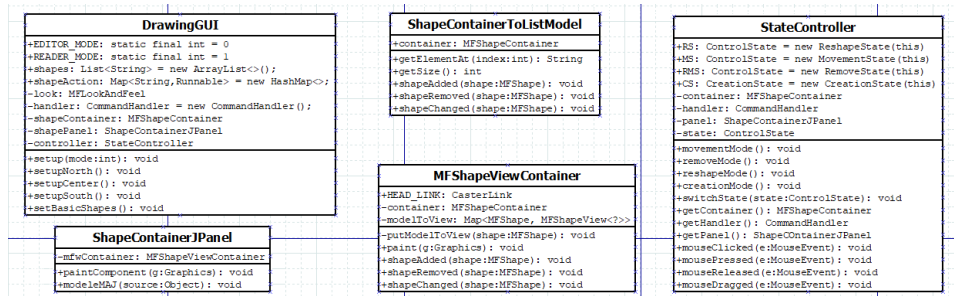


”utils” n’a qu’un fichier, ’GeoTools’, pour faire du raycasting et obtenir le vertex d’un polygone

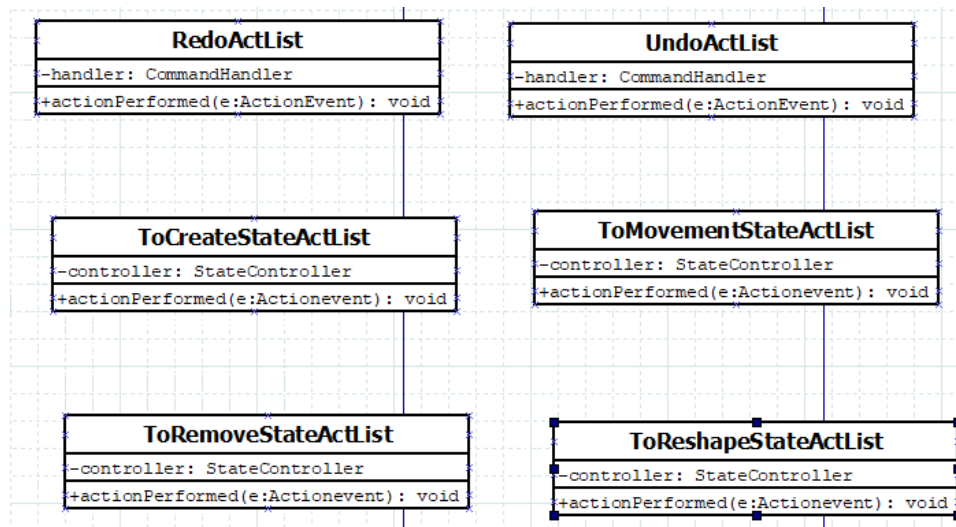


Il reste enfin le dossier ”view” qui represente graphiquement les action effectuées :

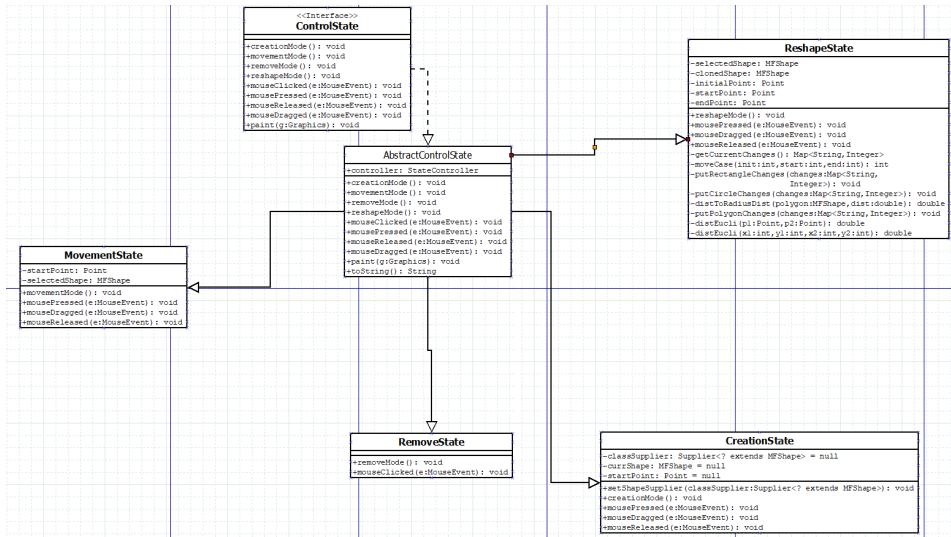
Les quelques fichiers present dans ce dossier permettent de configurer la vue, d’où le nom DrawingGUI et d’adapter le code à une version graphique.



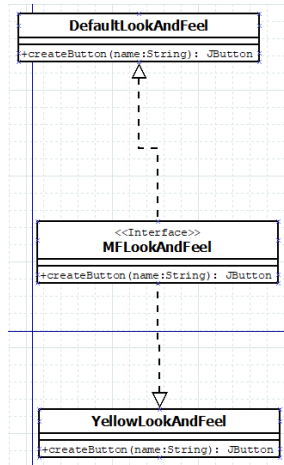
Plusieurs dossiers sont présents :
"action_listener" pour gérer l'annulation et la reproduction d'une action ainsi que les modifications qui peuvent être appliquées aux formes.



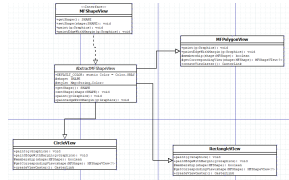
”control_state” pour créer les statuts des formes (est-ce qu’elles bougent, est-ce qu’elles sont redimensionnées,...)



”look_feel” pour mettre en place un look&feel



”shape_view” pour donner une representation aux formes dans la version graphique (couleur, taille,...)



4 Conclusion du projet

Ce projet est donc le resultat de plusieurs semaines de travail afin de mettre en pratique ce que nous avons appris durant ce semestre en java. Nous avons pu montrer l'étendu de notre apprentissage dans plusieurs domaines comme l'utilisation du modele MVC, l'interface graphique avec swing ou encore l'utilisation de design patterns. Nous sommes fiers du travail acoompli et de la version finale de notre jeu de memorisation de formes.